

Computer Security 2026 System Security

Roberto Guanciale

Memory bugs - effects

Access to Secret data

Corruption of program Data

Unexpected transfer of Control

Memory access Violation

Execution of code chosen by attacker

Binary Search

```
search(A, x, L, R):
```

```
    if L > R:
```

```
        return -1
```

```
    M = (L+R)/2
```

```
    if A[M] == x: return M
```

```
    if A[M] > x: return search(A, x, L, M-1)
```

```
    if A[M] < x: return search(A, x, M+1, R)
```

Row hammer

- side effect in dynamic random-access memory (DRAM)
 - memory cells interact electrically between themselves possibly discharging
- Hammering (reading multiple times) a memory line can flip a bit of another line
- Used in javascript for example to flip a bit of the JIT code, possibly a pointer
- Defense, ECC memories, physically separate memory on DRAM

Preventing Detecting Memory unsafety

— — —

Buffer overflow defenses

buffer overflows are widely exploited

- large amount of vulnerable code in use
- despite cause and countermeasures known

three defense approaches

- compile-time - harden new programs
- run-time - handle attacks on existing programs
- hardware-based defenses

Compile time Defenses: Language

— — —

use a modern high-level languages with strong typing

- you can not access untyped memory
- not vulnerable to buffer overflow

implementations combine compile-time restrictions, run-time checks, and memory management.

rust uses ownership and borrowing to enforce many lifetime rules without garbage collection.

- do have cost in performance, memory usage, and timing predictability
- hardware access and foreign interfaces may require trusted **unsafe** code
- guarantees depend on the correctness of unsafe components, libraries, compilers, and runtimes.

Compile time Defenses: safe coding

if using potentially unsafe languages e.g. C

- Check buffer capacity before every read or write.
- Validate lengths and check for integer overflow in size calculations.
- Handle allocation failures before using the returned pointer.
- Define ownership and ensure that accesses remain within an object's lifetime.
- For strings, account for the terminating null byte and handle truncation explicitly.
- use safer standard library variants
 - new functions, e.g. strncpy()
 - strncpy() can leave the destination without a terminating null byte.

Static analysis and formal verification

— — —

- Static analysis examines code without executing it and can identify potential defects.
- Formal verification proves specified properties using mathematical models.
- A proof establishes a guarantee
-

Expensive: ~2000\$ (2010) ~500\$ (2020) per line of code

Verified execution platforms

- isolation/separation kernels (seL4)
- software fault isolation (ARMor)
- tools such as Frama-C

Static analysis and formal verification

— — —

- Static analysis examines code without executing it and can identify potential defects.
- Formal verification proves specified properties using mathematical models.
- A proof establishes a guarantee **under explicit assumptions about the program and its environment.**

Expensive: ~2000\$ (2010) ~500\$ (2020) per line of code

Verified execution platforms

- isolation/separation kernels (seL4)
- software fault isolation (ARMor)
- tools such as Frama-C

Dynamic Verification (aka Testing)

- Testing executes a program with selected inputs.
- Fuzzing generates or mutates inputs to explore program behavior.
- Symbolic execution represents inputs symbolically and solves path constraints to generate test cases.
- Monitor program execution to identify bugs
 - Memcheck instruments the program to track
 - addressability: whether a memory location may be accessed.
 - definedness: whether a value has been initialized.

Dynamic Verification (aka Testing)

- Testing executes a program with selected inputs.
- Fuzzing generates or mutates inputs to explore program behavior.
- Symbolic execution represents inputs symbolically and solves path constraints to generate test cases.
- Monitor program execution to identify bugs
 - Memcheck instruments the program to track
 - addressability: whether a memory location may be accessed.
 - definedness: whether a value has been initialized.
- Testing can reveal defects in exercised executions.
Passing tests does not prove the absence of defects.

Background: HW based Memory protection

— — —

- Memory Management Unit
 - Page tables stored in memory
 - Configured by OS
 - Maps Virtual Addresses to Physical Addresses and access permissions
- Memory Protection Unit
 - Configuration in special registers
 - Configured by OS
 - Maps physical memory regions to access permissions

Run time defense: guard pages

- configured in Memory Management Unit as illegal addresses
- any access aborts process

can be placed between

- stack frames and heap buffers
- between critical regions of memory
 - between buffers of different network sessions
 - between network buffers and TLS certificates/JIT memory

Fat pointers

— — —

```
Buffer {  
    random key  
    data  
}  
Pointer {  
    key  
    base  
    bound  
    pointer  
}
```

Allocation:

- Generate a random key
- Return fat-pointer

Pointers arithmetic

- Check bounds

Dereference

- Check base key = pointer key

Free

- New random key for buffer

ARM Memory Tagging Extension

— — —

- Each 16-byte memory **granule** has a 4-bit allocation tag
- A pointer's logical tag occupies address bits 59–56.
- Memory Tag DB
 - 1 granule => 1 tag
 - Raw tag storage is 4 bits per 16 bytes: 3.125%.
 - For 16 GiB of tagged memory, 512 MiB of tag storage

MTE-aware malloc:

- Generate a random tag for the buffer, update the memory tag DB entries for **each** granule, return a tagged pointer

Memory Access (in HW)

- Check pointer tag = granule tag

MTE-aware free:

- Check pointer tag = granule tag, generate a new random tag for the buffer's granules

CHERI (e.g. RISC-V)

— — —

- A capability (used in place of plain pointers) contains an address, bounds, permissions, and additional metadata.
- Modern 64-bit CHERI-RISC-V uses 128-bit capabilities
- Hardware checks capability validity, bounds, and permissions on memory access.
- Programs can adjust the address and restrict authority. Derivation cannot expand the source capability's authority.
- Ordinary writes to stored capability bytes clear the associated validity tag.
- Instructions to copy CAP fields to general purpose registers
- Instructions to operate on CAP without expanding authority
 - increment base
 - decrease length
 - invalidate
- Load/store from CAP
- Jmp/Cjmp from CAP
- Preventing use-after-free additionally requires temporal-safety mechanisms.

Mitigating Memory unsafety

i.e., preserving some
properties of vulnerable
code

— — —

Control flow integrity

Prevent or detect alteration of the control flow due to

- Modification of return pointer
- Modification of a function pointer

Compile time Defenses: Stack protection

add entry and exit code to check stack for signs of corruption

- use random (different for every execution) canary
- e.g. GCC `-fstack-protector` (not all functions)
- Canaries were used in coal mines to detect the presence of carbon monoxide



Compile time Defenses: Stack protection

add entry and exit code to check stack for signs of corruption

- use random (different for every execution) canary
- e.g. GCC `-fstack-protector` (not all functions)

check for overwrite between local variables and saved frame pointer and return address

- abort program if change found
- issues: recompilation, debugger support

Compile time Defenses: Stack protection

add entry and exit code to check stack for signs of corruption

- use random (different for every execution) canary

Stack:

Return ptr

Frame ptr

Local var1

Local var2

Param 1

Param 2

Compile time Defenses: Stack protection

add entry and exit code to check stack for signs of corruption

- use random (different for every execution) canary

Stack:

Return ptr

Frame ptr

1243152311653 // Canary

Local var1

Local var2

Param 1

Param 2

Compile time Defenses: Stack protection

— — —

- Store return addresses in a separate shadow stack.
- On return, compare against or restore the protected return address.
- Protect the shadow stack against writes from vulnerable program code.
- Shadow stacks protect returns. They do not protect arbitrary function pointers or application data.

Shadow Stack

Guard page

...

Stack

Guard page

...

Heap

...

Run time Defenses: Target address encryption

— — —

- Indirect jumps (e.g. jumps to non-constants) are necessary
 - Function return
 - Callbacks, Virtual methods, Exception handling
- Encrypt the addresses
 - Notice xor is not a good way of doing encryption

```
int (*func1)(int) = double;  
  
func1(15);
```

====>

```
int (*func1)(int) = double ^ key;  
  
(func1 ^ key)(15);
```

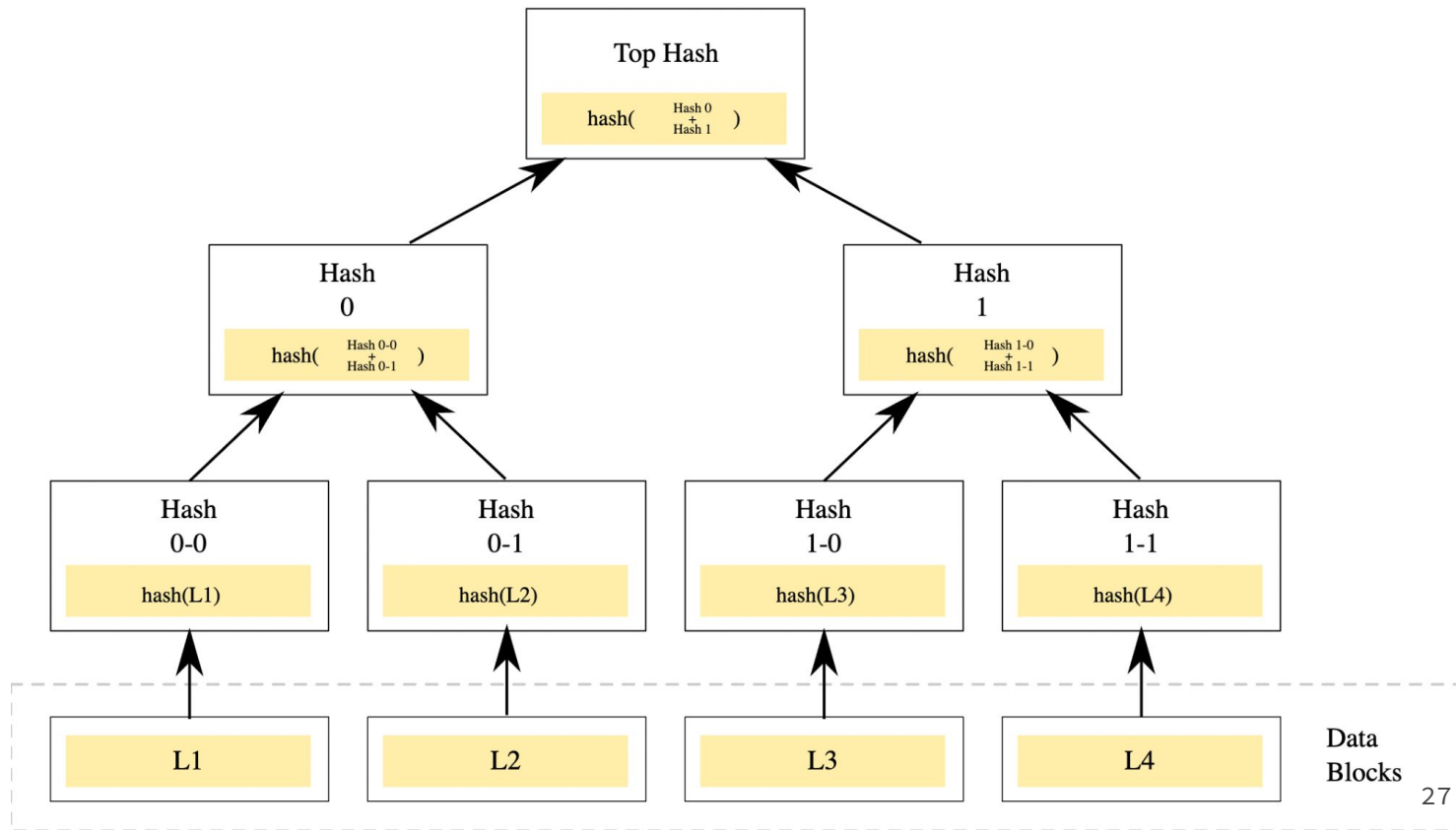
Run time Defenses: Control Flow Attestation

— — —

- Provide authenticated evidence of an execution control-flow path for a verifier to check
- Alter the system to keep track of PC updates
- To update the measurement:
 - HW modifications
 - Static analysis of no indirect jmp, all indirect jumps are made by a trusted SW via a trampoline
- To keep track of the measurements
 - Only hash
 - Isolated memory

```
int (*func1)(int) = double;  
func1(15);  
  
====>  
  
int (*func1)(int) = double;  
invoke(func1, 15);  
invoke(f,args) {  
    log(f);  
    (f)(args);  
}
```

Merkle tree



Code integrity

— — —

- Prevention, detection, mitigation of code injection
- Due to a buffer controlled by the attacker (e.g. where a network packet is stored) being executed
- Due to existing code being overwritten

Code integrity: Executable Address Space Protection

— — —

- use virtual memory support to make some regions of memory non-executable
 - e.g. stack, heap, global data need HW support in MMU
- long existed on SPARC / Solaris systems
- recent on x86/ARM Linux/Unix/Windows systems

STACK	R/W
HEAP	R/W
Global Data	R/W
Code	R/X

Code integrity: Executable Address Space Protection W $\oplus$ X

— — —

- use virtual memory support to make some regions of memory non-executable
 - e.g. stack, heap, global data need HW support in MMU
- long existed on SPARC / Solaris systems
- recent on x86/ARM Linux/Unix/Windows systems
- issues: support for executable stack/heap code
 - needed for JIT (e.g. Java) or nested functions
 - need special provisions
 - `mprotect(ptr, size, (PROT_READ | PROT_EXEC));`
- Attacker can
 - Inject payload
 - Corrupt control flow to invoke `mprotect` (see ROP)
 - Execute the payload

Code integrity: code signature

— — —

- Before executing a program, check that it satisfies the platform's trust policy, usually by hashing its binary

Options:

- Keep protected database of valid digests, check that $\text{SHA}(\text{program})$ in DB
- Use program with certificate
 - $\text{signature} = \text{Enc}_{\text{MSsk}}(\text{SHA}(\text{program}))$
 - Check $\text{Dec}_{\text{MSpu}}(\text{signature}) = \text{SHA}(\text{program})$
 - Chain of certificates can be used to prove authorship and integrity
- Problem
 - Pages are usually loaded dynamically, therefore expensive for entire program

Code integrity: malware detection

— — —

- Before executing a program, check that it satisfies the platform's trust policy, usually by hashing its binary

Options:

- Keep a database of binary fragments of well known malwares, check that $\text{Intersect}(\text{program}, \text{db}) = \text{empty}$
 - Look at the slides on Malware for problems related to polymorphic viruses

Code integrity: Run-time code signature

— — —

- Enforce $W\oplus X$ policy
- Check signature whenever a page becomes executable (i.e. `mprotect`)

Options for signature checks

- Keep protected database of valid digests, check that $\text{SHA}(\text{page})$ in DB
- Use program with certificate
 - $\text{signature} = \text{Enc}_{\text{MSsk}}(\text{SHA}(\text{page}))$
 - Check $\text{Dec}_{\text{MSpu}}(\text{signature}) = \text{SHA}(\text{page})$
 - Chain of certificates can be used to prove authorship and integrity
- Keep a database of binary fragments of well known malwares, check that $\text{Intersect}(\text{page}, \text{db}) = \text{empty}$
- JIT compilation requires a controlled policy for generating and enabling executable code.
 - See MProtect

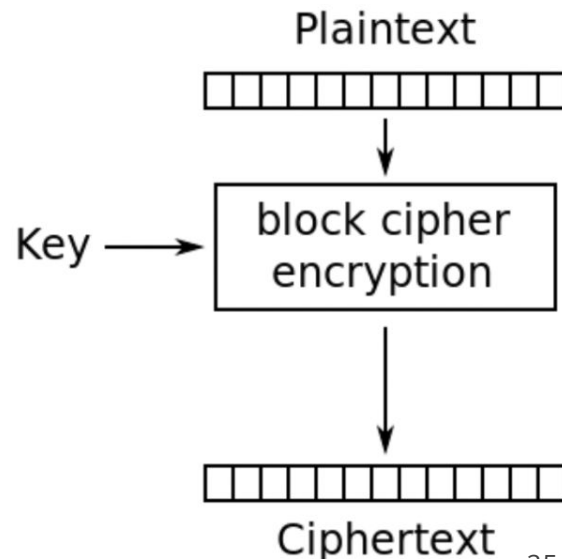
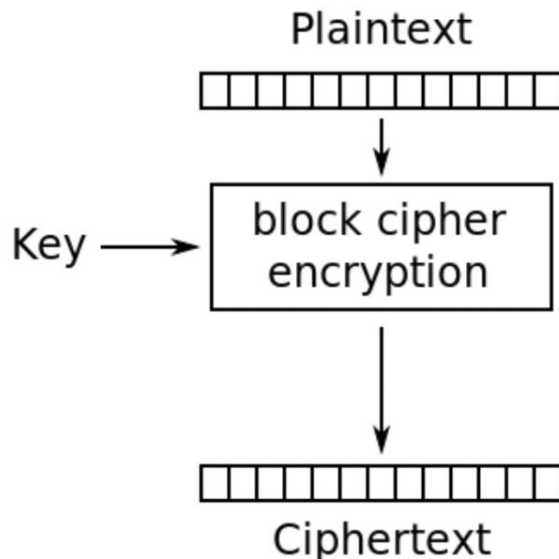
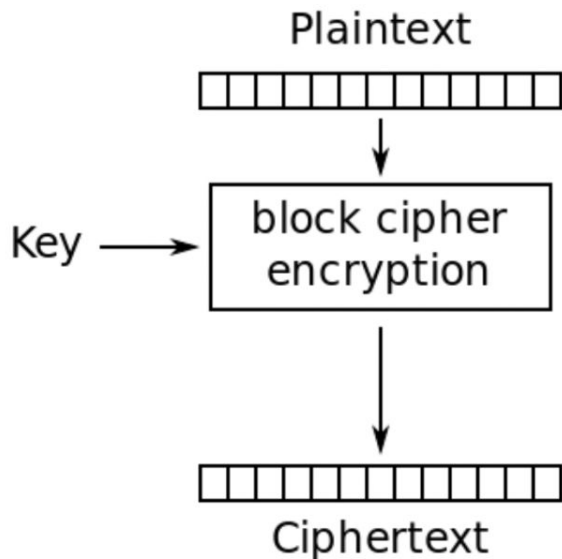
Related topic: Disk Encryption

— — —

Would you use symmetric or
asymmetric encryption?

Disk Encryption

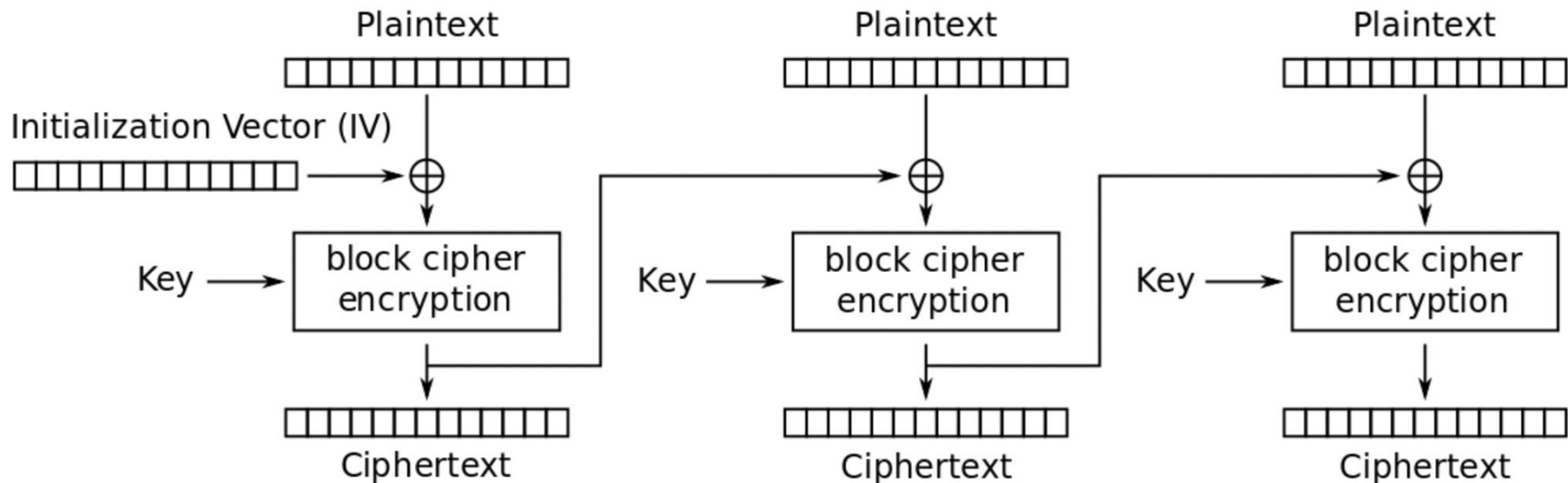
Is it secure to use AES ECB (Electronic codebook)?



Disk Encryption

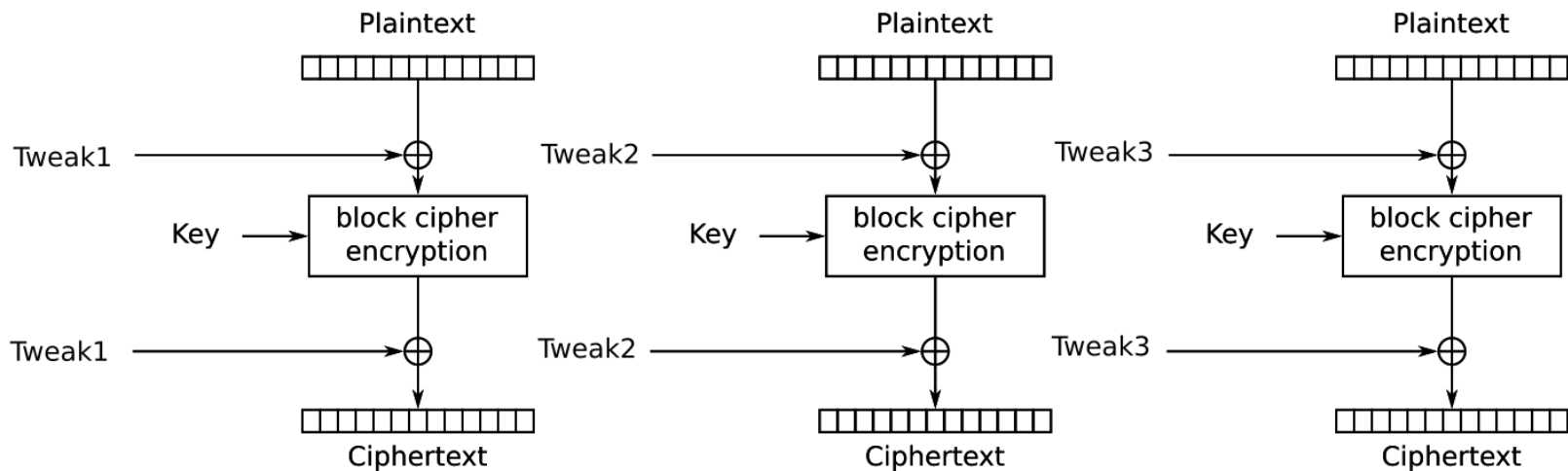
Is it secure to use AES ECB (Electronic codebook)?

Can we use Cipher Block Chaining?



Disk Encryption

Tweakable Encryption (For integrity we also need a MAC)



Xor Encrypt Xor (XEX) mode encryption

Instruction Set Randomization

— — —

Make every Process's CPU unique

If the attacker does not know the
target ISA, it is difficult to
produce injectable code

Instruction Set Randomization

— — —

Make every Process's CPU unique

If the attacker does not know the target ISA, it is difficult to produce injectable code

Code is encrypted in memory:

$M[pc] = \text{Encrypt}(\text{instruction})$

Execution Of Instruction:

fetch $M[pc] = c$

$i = \text{Decrypt}(c)$

execute (i)

If attacker injects c' he cannot predict how c' will be decrypted (assuming attacker does not know key)

Decryption requirements

— — —

1. Cheap
Symmetric block ciphers
2. Preserve instruction length
No MAC
3. Support random accesses
No Output feedback (OFB)
4. Use Tweakable Encryption

Making life of attacker harder

— — —

Diversification

— — —

Counter attacks by making difficult for the attacker to predict the results of their activities

Run-time Defenses: Address Space Randomization

— — —

- ASLR randomizes the locations of memory mappings.
- Linux can randomize the stack, shared libraries, other mappings, and heap placement.
- Position Independent Executables (PIE) also support randomization of the main executable.
- Different regions need not share one random displacement.
- Offsets within a loaded image generally remain fixed.
- Address disclosures can undermine protection.
- To disable it: `echo 0 | sudo tee /proc/sys/kernel/randomize_va_space`

Run-time Defenses: Base Stack Randomization

— — —

- The OS varies stack placement when creating a new program image.
- The stack pointer tracks the current stack position.
- Randomization makes absolute stack addresses harder to predict.
- Relative offsets within a stack frame may remain predictable.
- Stack placement randomization generally requires no recompilation

Run-time Defenses: Base Stack Randomization

— — —

```
void f() {  
    char [16] pwd;  
    char [16] input;  
    ...  
    gets(input);  
}
```

Effective to counter modifications pwd?

Run-time Defenses: Base Stack Randomization

— — —

```
void f() {  
    char [16] buffer;  
    gets(buffer)  
}  
  
void nuke() {  
  
}
```

Effective to counter redirection to
nuke?

```
void f() {  
    char [16] buffer_for_shell;  
    gets(buffer_for_shell)  
}
```

Effective to counter execution of
shell in buffer?

Run-time Defenses: Heap Randomization

— — —

- Heap addresses depend on OS mappings and allocator behavior.
 - `ptr = malloc(1024);`
- ASLR varies base address and mapping locations.
- Some allocators add randomization to object placement or reuse.
- ASLR does not imply a random order of individual allocations, but that their addresses are difficult to predict
- Randomization does not prevent use-after-free or double-free bugs
- Randomization does not affect iter data structure layout

```
struct {  
    char x[10];  
    char y[10];  
}
```

Run-time Defenses: Global Randomization

— — —

- Globals in a PIE executable move with that executable's load address.
- Their relative offsets generally remain fixed.
- Position-independent code and relocations support loading at different bases.
- Randomizing individual globals requires additional transformations.

```
X = MEM[1MB]
```

```
==>
```

```
offset = MEM[offset-add]
```

```
X = MEM[1MB + offset]
```

Run-time Defenses: ASLR

— — —

- PIE allows the main executable to load at varying addresses.
- Function and gadget addresses change with the load base.
- An address disclosure (e.g. side channels) may reveal that base.
- ASLR complicates exploitation but does not enforce valid control flow.
- Programs must use location independent code

1 MB: jmp 1.2 MB ==> 2 MB: jmp PC + 0.2 MB

1.2 MB: code ==> 2.2 MB: code

Compile-time Defenses:

Use polymorphic technique of malware

— — —

every instance of the application is different

- different order of arguments

```
int memcpy(dst, src, size) {...}
```

```
memcpy(dst, src, 1024);
```

=>

```
int memcpy(size, src, dst) {...}
```

```
memcpy(1024, src, dst);
```

Compile-time Defenses:

Use polymorphic technique of malware

— — —

every instance of the application is different

- different order of arguments
- different number of local variables

```
int x = y + 20;
```

=>

```
int z = 20;
```

```
int x = y+z;
```

Compile-time Defenses:

Use polymorphic technique of malware

— — —

every instance of the application is different

- different order of arguments
- different number of local variables
- different alignment of data-structures

```
struct book {char [100] title, author * x};
```

=>

```
struct book {author * x, char [10] dummy, char [110] title};
```

Compile-time Defenses:

Use polymorphic technique of malware

— — —

- Use of intermediate languages (e.g. LLVM)
- C program is compiled to IR (e.g. using CLANG)
- **IR is transformed to add randomization**
- IR is optimized
- IR is compiled to machine language

**Be prepared for
the worst**

— — —

Isolation

— — —

- Memory safety does not eliminate logic errors or all security vulnerabilities.
- It is difficult to write bug free code
 - <https://www.vidarholen.net/contents/wordcount>
- The trusted computing base contains the components whose correct behavior is necessary for a security property.
- Reduce this trusted base and give each component only the authority it needs.
- Isolate critical components from faults or compromise in less trusted components.
- Isolation depends on correct enforcement and controlled communication across boundaries.

Isolation: Smart thermostat

— — —



- Components
 - Safety controller: reads temperature and decides the heater state.
 - Sensor and heater drivers: access the physical devices.
 - Network service: receives remote temperature requests. (can be compromised)
 - ML optimizer: proposes an energy-saving temperature. (can be compromised)
 - Logger: records events.
- Required control policy
 - At or below 15°C, request heating.
 - At or above 25°C, request no heating.
 - Between these limits, follow a validated comfort preference.
 - Evaluate the policy every second, even when ML or Network Services are compromised

Isolation: Smart thermostat

— — —



- Safety limits are fixed trusted configuration
- Use bounded queues, bounded request processing, reserved control-loop CPU time, and non-blocking logging.
- Safety controller
 - Read sensor, command heater via drivers
 - Receive and validate (bounded) target requests, publish status via ML and Network
 - Valid network requests override ML suggestions.
 - Log events
- Drivers
 - Access only their assigned devices;
 - All sensors and actuators: accept commands only from the controller
 - NIC driver: accept commands only from the network service
- Network service
 - Submit target requests; read limited status via safety controller
 - Log user requests
- ML optimizer
 - Read approved temperature history, submit target suggestions via safety controller
 - Log AI decisions
- Logger
 - Receive events; write its own log storage

Isolation: Smart thermostat

— — —



- Write heater registers directly
 - Blocked. The network service cannot map device registers or invoke the heater driver's restricted command interface.
- Request 5°C
 - Rejected by the controller. Validation remains effective even after the network service is compromised.
- Send a million requests
 - Bounded impact. Excess requests are dropped Bounded processing and scheduling isolation preserve the control loop.
- Stop sending requests
 - Control continues. The controller independently requests heating at or below 15°C.
- Repeatedly request 24°C while ML requests 18°C
 - Allowed within the policy. User requests take precedence, so the compromised network service can affect comfort and energy use. Isolation preserves the safety policy, not the authenticity of user intent through a compromised service.
- Modify a log entry
 - Direct modification is blocked. The network service can only append to log storage. However, if not bounded can lead to a DOS

Microkernels

— — —

- L4 and Minix is the most famous

“A concept is tolerated inside the microkernel only if moving it outside the kernel, i.e., permitting competing implementations, would prevent the implementation of the system's required functionality”

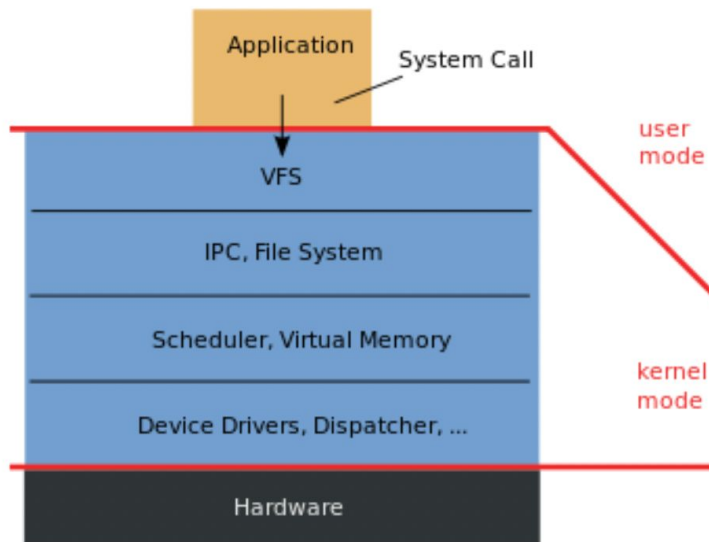
- address spaces
- Threads
- Scheduling
- inter-thread communication

Everything else is outside the kernel (e.g. drivers) 15 thousands lines of code

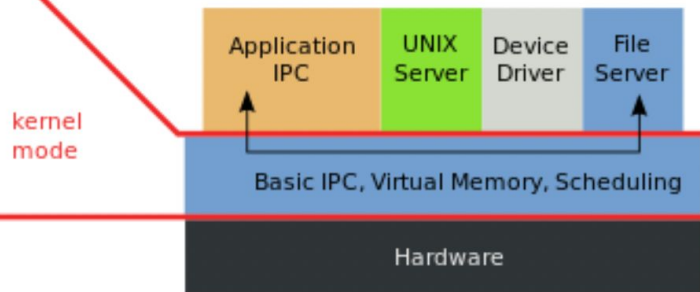
Linux 30 M Loc vs seL4 30 K Loc

Microkernels

Monolithic Kernel
based Operating System



Microkernel
based Operating System



Software Fault Isolation

— — —

Sandbox non-critical code

Google Chrome Native Client
(discontinued)

Modify binary to ensure that
overflows can not access critical
resources

E.g. everything above 0x01000000 is
critical everything below
0x00FFFFFF is non-critical

```
mem[x] := y
```

```
=====>
```

```
x := x & 0x00FFFFFF
```

```
mem[x] := y
```

Software Fault Isolation

— — —

Sandbox non-critical code

Google Chrome Native Client
(discontinued)

Modify binary to ensure that
overflows can not access critical
resources

E.g. everything above 0x01000000 is
critical everything below
0x00FFFFFF is non-critical

```
mem[x] := y
```

```
x+=1
```

```
mem[x] := y
```

```
=====>
```

```
x := x & 0x00FFFFFF
```

```
mem[x] := y
```

```
x+=1
```

```
x := x & 0x00FFFFFF
```

```
mem[x] := y
```

Software Fault Isolation

— — —

Sandbox non-critical code

Google Chrome Native Client
(discontinued)

Modify binary to ensure that
overflows can not access critical
resources

E.g. everything above 0x01000000 is
critical everything below
0x00FFFFFF is non-critical

```
mem[x] := y
```

```
x+=1
```

```
mem[x] := y
```

```
=====>
```

```
x := x & 0x00FFFFFF
```

```
mem[x] := y
```

```
x+=1
```

```
mem[x] := y
```

Hypervisors

— — —

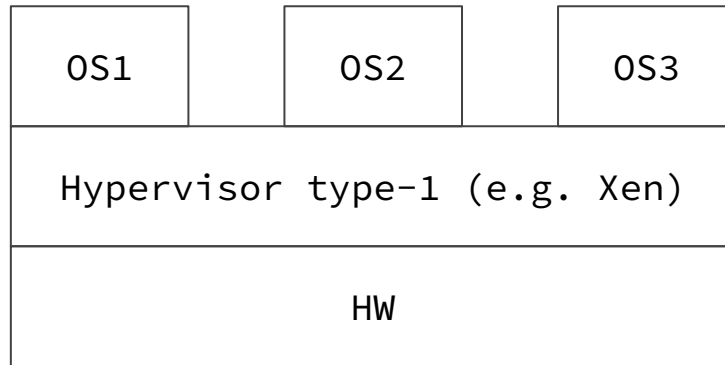
Execute below OS

Isolate complete OSes from each other

Can inspect the behavior of a
(possibly) buggy OS

Today hypervisors use HW support:

- Stage 1 MMU configured by OS and maps Virtual Addresses to Intermediate Addresses
- Stage 2 MMU configured by Hypervisor and maps Intermediate Addresses to Physical Addresses



Hypervisors

— — —

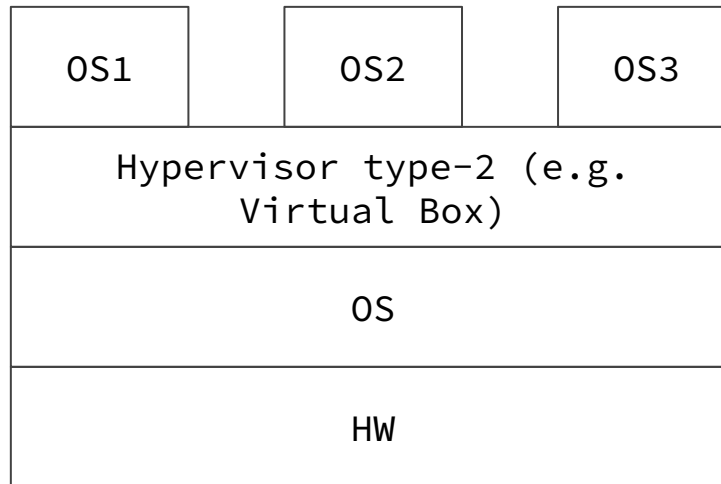
Execute below OS

Isolate complete OSes from each other

Can inspect the behavior of a
(possibly) buggy OS

Today hypervisors use HW support:

- Stage 1 MMU configured by OS and maps Virtual Addresses to Intermediate Addresses
- Stage 2 MMU configured by Hypervisor and maps Intermediate Addresses to Physical Addresses



Hypervisors

— — —

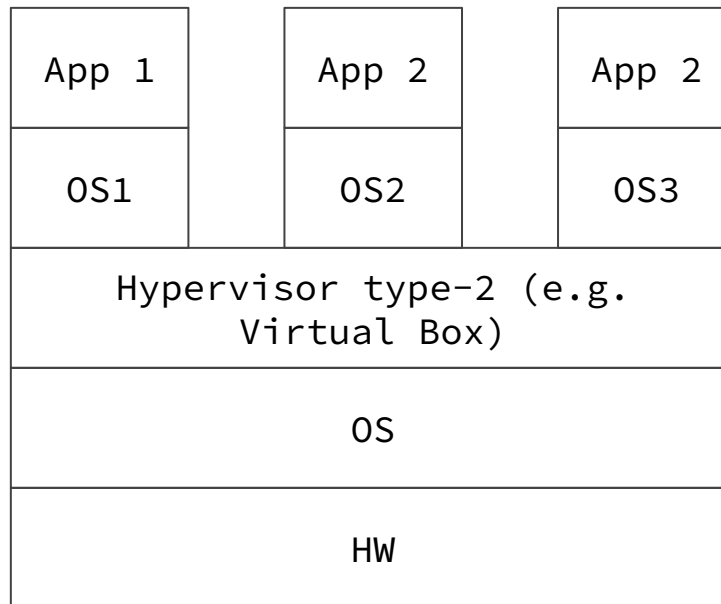
Execute below OS

Isolate complete OSes from each other

Can inspect the behavior of a
(possibly) buggy OS

Today hypervisors use HW support:

- Stage 1 MMU configured by OS and maps Virtual Addresses to Intermediate Addresses
- Stage 2 MMU configured by Hypervisor and maps Intermediate Addresses to Physical Addresses

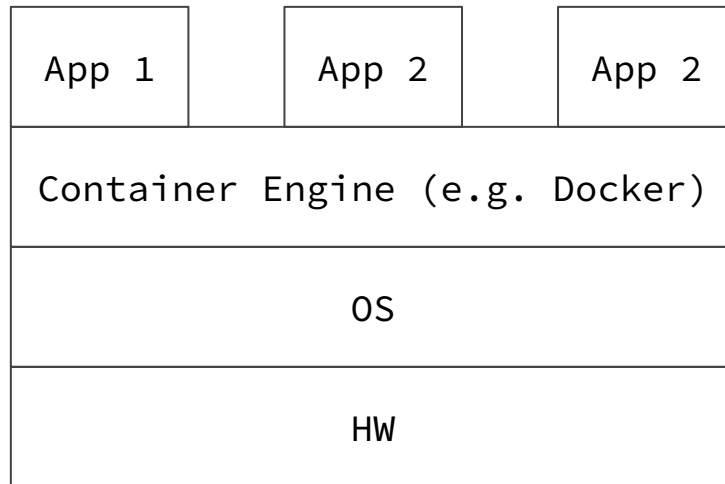


Containers

— — —

Virtualise OS

Reuse the same OS



Containers

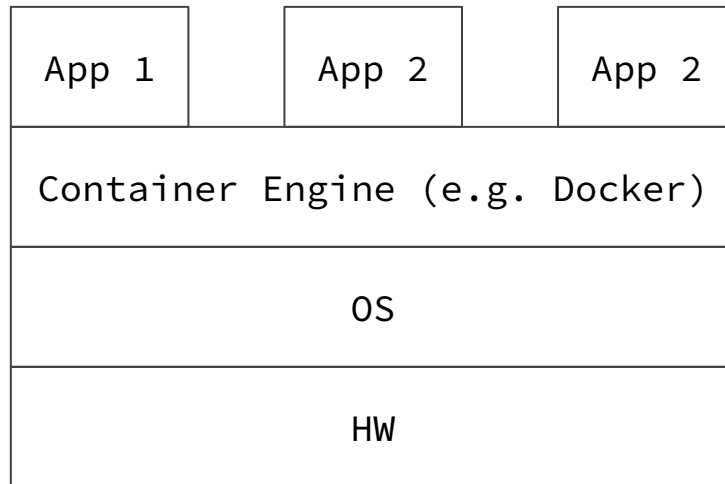
— — —

Virtualise OS

Reuse the same OS

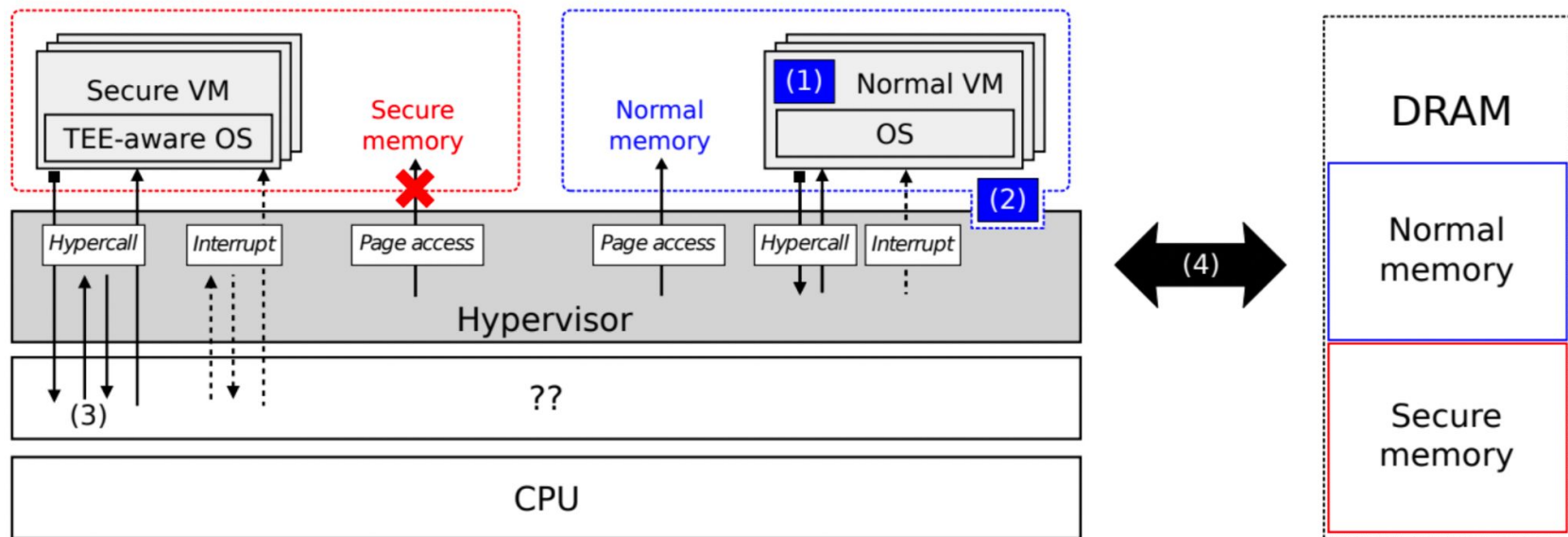
Run-time monitor (also for hypervisors)

- checking code signature
- Behavioral monitoring
- Resource usage analysis
- Quarantine
- Honeypots



Trusted Execution Environments

Intel SGX, ARM trustzone, Intel TDX, ARM CCA, AMD SEV



Trusted Execution Environments

— — —

Intel SGX, ARM trustzone, Intel TDX, ARM CCA, AMD SEV

- Intel SGX: protects application enclaves.
- Arm TrustZone: separates Secure and Non-secure execution.
- Intel TDX, Arm CCA Realms, AMD SEV variants: provide mechanisms for protecting virtual machines.

The protected unit, trusted components, and supported attack model differ between technologies.

Trusted Execution Environments

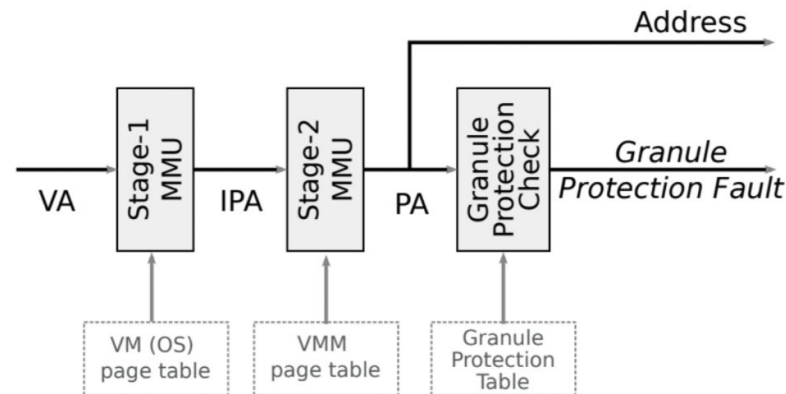
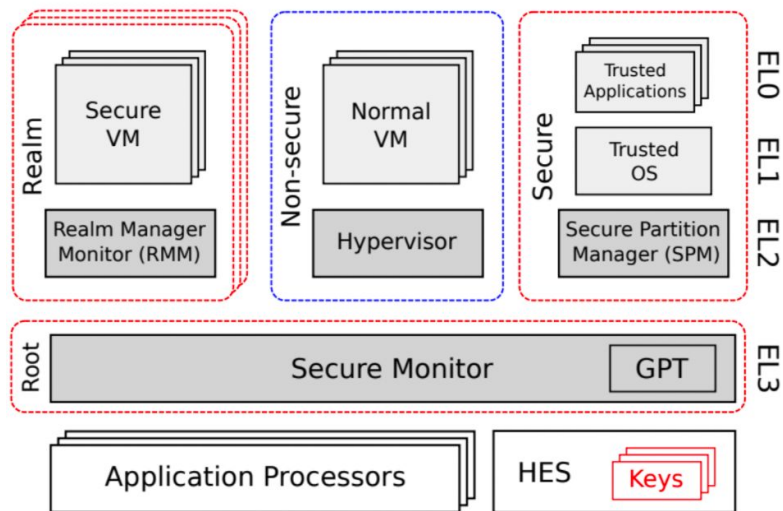
— — —

Intel SGX, ARM trustzone, Intel TDX, ARM CCA, AMD SEV

- Memory encryption, integrity protection, and device-access protection vary by technology.
- Do not trust hypervisor (and usually owner of the physical machine)
 - Also provide some sort of memory encryption to prevent physical attacks or attacks from devices
- Resource management delegated to hypervisor (e.g., no protection against DOS)
- Provide some mechanism for remote attestation
 - Allow a trusted component to “measure” (e.g., digest of executing code and configuration)
 - Allow to establish secure channel via crypto
 - Allow to report measurements

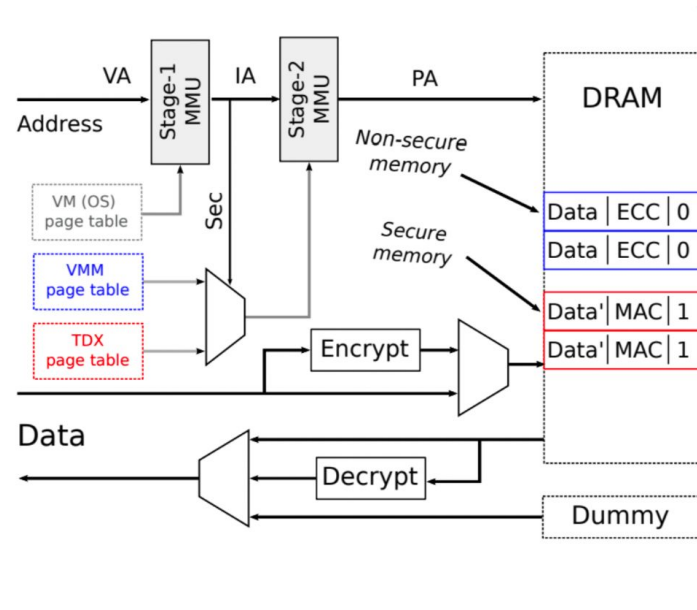
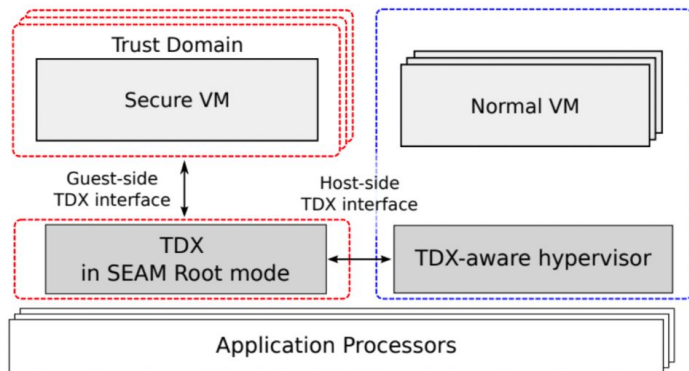
Trusted Execution Environments

Intel SGX, ARM trustzone, Intel TDX, **ARM CCA**, AMD SEV



Trusted Execution Environments

Intel SGX, ARM trustzone, **Intel TDX**, ARM CCA, AMD SEV



What each defense guarantees

Defense	Guarantee	Key assumptions	Remaining attacks
Memory-safe language	Prevents invalid memory accesses in safe code	Sound implementation and trusted unsafe interfaces	Logic errors, side channels, unsafe-code bugs
Stack canary	Detects changes to the canary before a protected return	Canary remains secret; overwrite crosses it; check executes	Canary disclosure, writes that bypass it, data corruption
W^oX	Prevents writable and executable mappings of the same memory under the enforced policy.	Correct page permissions and control of writable aliases	ROP, data-only attacks, abuse of permission changes
ASLR	Makes target addresses harder to predict	Sufficient randomness and no useful address disclosure	Information leaks, guessing, relative overwrites
CFI / shadow stack	Restricts indirect targets / protects return addresses	Complete enforcement and protected metadata	Abuse of allowed targets, data-only attacks
Isolation / sandbox	Restricts a compromised component's access	Correct enforcement and limited permissions	Abuse of permitted operations, interface bugs, sandbox escape